Individual Project: Mobile Application Final Project Bakery App

(With Room Database)

Course Name: MOBI3002/3091

Instructor: Kalpana Samaraweera Mudalige

Student: Kael Payette

Date: 2025-12-08

Table of Contents

1. Project Objective ..... ……..3

2. Solution Use case Diagrams……3

3. Solution Logical ERD ..... ……4

4. Program Code 5

5. Screenshots 38

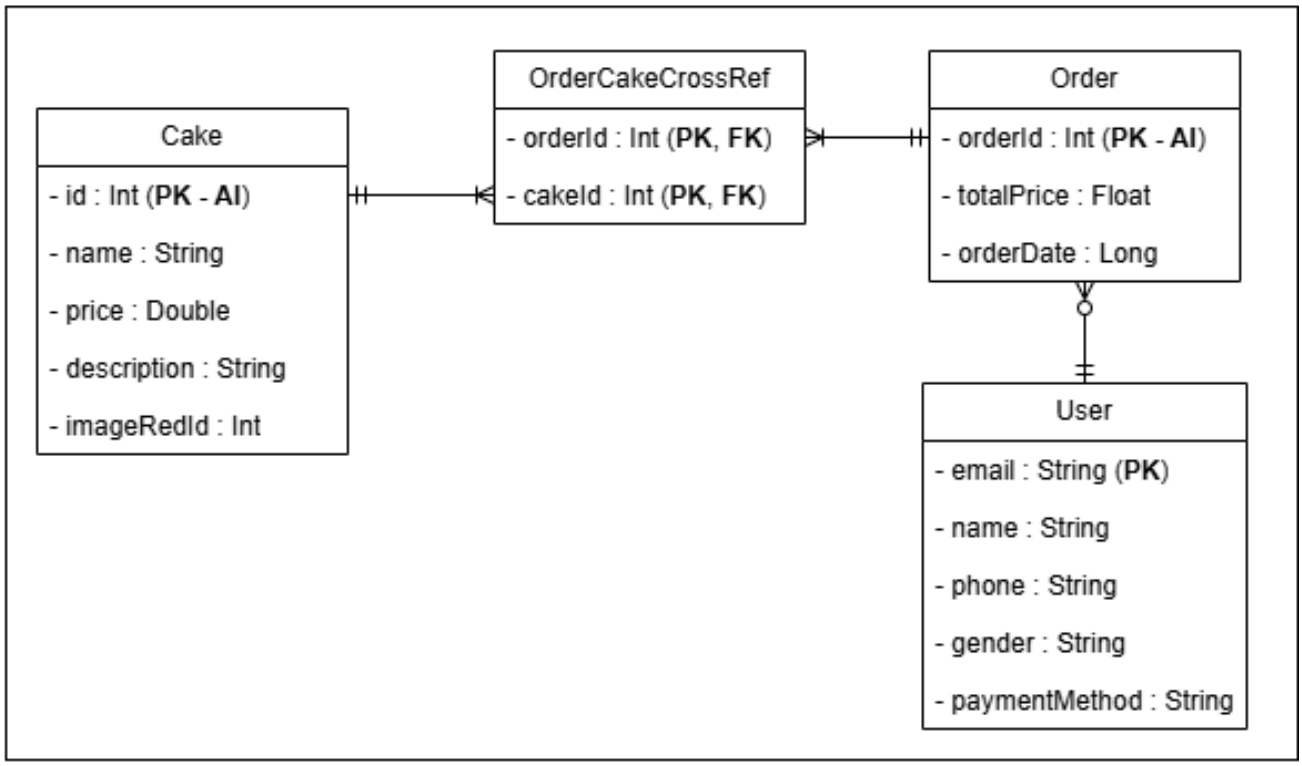
6. Code Explanation/Justifications.....40

7. References 41

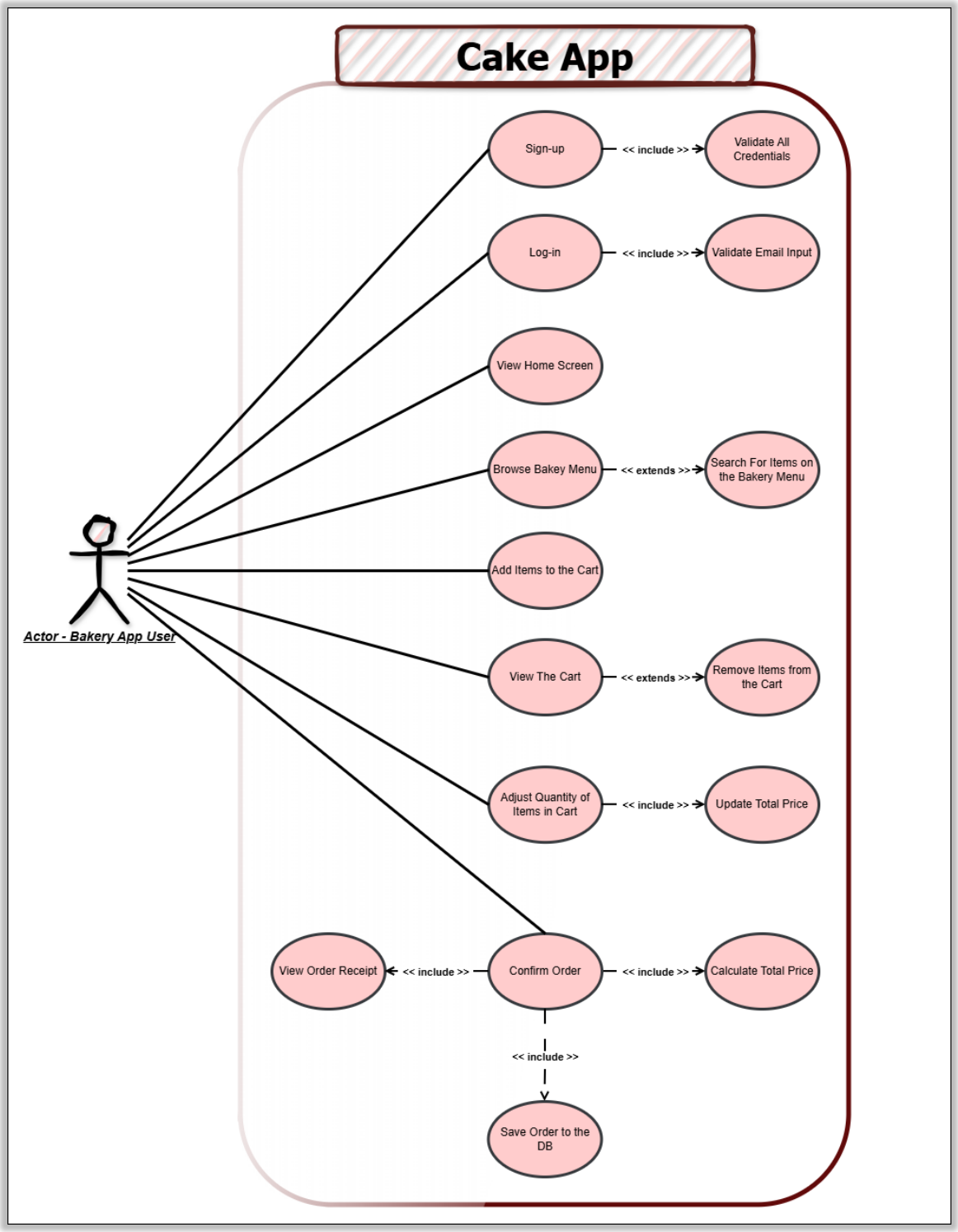
1. Project Objective

This individual project aims to develop a fully functional mobile application prototype using Android. You will design, develop, and deploy a mobile application that demonstrates core Android development skills, including app lifecycle management, Kotlin programming, UI design with Jetpack, event and gesture handling, data persistence, CRUD operations, and MVVM architecture with proper error handling. The project will conclude with deploying and distributing the app using the most appropriate method. The project's main objective is to showcase individual proficiency in developing a complete mobile solution for a real world scenario.

2. Solution Logical ERD



3. Solution Use Case Diagram



4. Program Code

4.1. DAOs

CakeDao.kt:

```
package com.example.ia4_bakery_kp.data.dao

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import com.example.ia4_bakery_kp.data.entity.Cake

@Dao
// DAO interface for accessing 'cakes' table
interface CakeDao {
    // Insert cakes into the database, replace if duplicate
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertCake(cake: Cake)

    // Show the entire cake menu, all cakes in the database - if
nothing is in the search bar this is what will be shown
    @Query("SELECT * FROM cakes")
    suspend fun getAllCakes(): List<Cake>

    // Search bar query, finds all desserts where the 'name' contains
the users search term
    // non-case sensitive, which makes it more user friendly
    @Query("SELECT * FROM cakes WHERE name LIKE '%' || :search || '%'")
    // Selects all from the cakes table
    // where the name is like whatever the user searches/types
    // The first '%' = allows any characters before the users search
term
    // The second '%' = allows any characters after the users search
term
    // '||' = concatenates the search string with the 2 '%'
    suspend fun searchCakes(search: String): List<Cake>
}
```

OrderDao.kt:

```
package com.example.ia4_bakery_kp.data.dao

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Transaction
import com.example.ia4_bakery_kp.data.entity.Order
import com.example.ia4_bakery_kp.data.entity.OrderCakeCrossRef
import com.example.ia4_bakery_kp.data.entity.OrderWithCakes

@Dao
interface OrderDao {
```

```
    // Insert a new order, replaces existing order if duplicate
'orderId' exists
    // Returns the ID of the inserted order as a 'Long'
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertOrder(order: Order): Long

    // Insert join table entries linking orders to cakes
    // Replaces existing order-cake pairs if duplicates are found
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertOrderCakeCrossRef(crossRefs:
List<OrderCakeCrossRef>)

    // Fetch one order and all associated cakes using the
OrderWithCakes relation
    // '@Transaction' ensures consistency when loading related entities
    @Transaction
    @Query("SELECT * FROM orders WHERE orderId = :orderId")
    suspend fun getOrderWithCakes(orderId: Int): OrderWithCakes
}
```

UserDao.kt:

```
package com.example.ia4_bakery_kp.data.dao

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import com.example.ia4_bakery_kp.data.entity.User

@Dao
// DAO interface for accessing 'users' table
interface UserDao {

    // Insert a new user into the table, replaces existing user if the
email already exists
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertUser(user: User)

    // Find a user by email, returns the email if found or null if no
user is found
    @Query("SELECT * FROM users WHERE email = :email LIMIT 1")
    suspend fun getUserByEmail(email: String): User?
}
```

4.2. Entities

Cake.kt:

```
package com.example.ia4_bakery_kp.data.entity
```

```
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "cakes") // cakes table: stores Cake entities
data class Cake(
    @PrimaryKey(autoGenerate = true) // Autoincrement the primary key
'id'
    val id: Int = 0, // Simple fields for cakes, has since been updated
to include multiple type of treats/desserts
    val name: String,
    val price: Double,
    val description: String,
    val imageResId: Int // Holds the images resource ID, much more
simple than storing each picture in the database
)
```

Order.kt:

```
package com.example.ia4_bakery_kp.data.entity

import androidx.room.Entity
import androidx.room.PrimaryKey
import java.util.Date

@Entity(tableName = "orders") // 'orders' table: stores Order entities
data class Order(
    @PrimaryKey(autoGenerate = true)
    val orderId: Int = 0, // Autoincrement the primary key 'orderId'
    // Simple fields for order, will be displayed in the receipt
    val totalPrice: Float,
    val orderDate: Date
)
```

User.kt:

```
package com.example.ia4_bakery_kp.data.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "users") // 'users' table: stores User entities
data class User(
    // The primary key is the user's email, as it should be unique for
every user
    @PrimaryKey val email: String,

    // Create fields for the login form data
    val name: String,
    val phone: String,
    val gender: String,
    val paymentMethod: String
)
```

OrderCakeCrossRef.kt:

```
package com.example.ia4_bakery_kp.data.entity

import androidx.room.Entity

@Entity(primaryKeys = ["orderId", "cakeId"]) // Join table linking
'orders' to 'cakes', composite primary keys prevent duplicates in the
table
// Aka each order-cake pair is unique
// cakes and orders have a many-to-many relationship
data class OrderCakeCrossRef(
    val orderId: Int,
    val cakeId: Int
)
```

OrderWithCakes.kt:

```
package com.example.ia4_bakery_kp.data.entity

import androidx.room.Embedded
import androidx.room.Junction
import androidx.room.Relation

class OrderWithCakes { // Represents an 'Order' plus its 'Cakes'
    @Embedded
    lateinit var order: Order // Embedded 'Order' entity (base order
details)

    @Relation( // '@Relation' links orderId to cakeId via
OrderCakeCrossRef join table
        parentColumn = "orderId",
        entityColumn = "id",
        associateBy = Junction(
            value = OrderCakeCrossRef::class,
            parentColumn = "orderId",
            entityColumn = "cakeId"
        )
    )
    lateinit var cakes: List<Cake> // Provides list of cakes belonging
to this order
}
```

4.3. Repositories

CakeRepository.kt:

```
package com.example.ia4_bakery_kp.data.repository

import com.example.ia4_bakery_kp.data.dao.CakeDao
import com.example.ia4_bakery_kp.data.entity.Cake

class CakeRepository(private val cakeDao: CakeDao) {
```

```
    // Insert a new cake into the database
    suspend fun insertCake(cake: Cake) {
        cakeDao.insertCake(cake)
    }

    // Get all cakes from the database
    suspend fun getAllCakes(): List<Cake> {
        return cakeDao.getAllCakes()
    }

    // Search cakes by name, not case sensitive
    suspend fun searchCakes(search: String): List<Cake> {
        return cakeDao.searchCakes(search)
    }
}
```

OrderRepository.kt:

```
package com.example.ia4_bakery_kp.data.repository

import com.example.ia4_bakery_kp.data.dao.OrderDao
import com.example.ia4_bakery_kp.data.entity.Order
import com.example.ia4_bakery_kp.data.entity.OrderCakeCrossRef

class OrderRepository(private val orderDao: OrderDao) {

    // Add an order with associated cakes:
    suspend fun addOrder(order: Order, cakeIds: List<Int>): Long {
        val orderId = orderDao.insertOrder(order)
        val crossRefs = cakeIds.map { cakeId ->
            OrderCakeCrossRef(orderId = orderId.toInt(), cakeId =
cakeId)
        }
        orderDao.insertOrderCakeCrossRef(crossRefs)
        return orderId
    }

    // Get an order with its associated cakes:
    suspend fun getOrderWithCakes(orderId: Int) =
orderDao.getOrderWithCakes(orderId)
}
```

UserRepository.kt:

```
package com.example.ia4_bakery_kp.data.repository

import com.example.ia4_bakery_kp.data.dao.UserDao
import com.example.ia4_bakery_kp.data.entity.User

class UserRepository(private val userDao: UserDao) {

    // Register a new user (insert the user into the database)
    suspend fun registerUser(user: User) {
        userDao.insertUser(user)
```

```
    }

    // Login: check if a user exists by email, returns the user if
found or null if not
    suspend fun loginUser(email: String): User? {
        return userDao.getUserByEmail(email)
    }
}
```

4.4. Data

AppDatabase.kt:

```
package com.example.ia4_bakery_kp.data

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase
import androidx.room.TypeConverters
import com.example.ia4_bakery_kp.data.dao.CakeDao
import com.example.ia4_bakery_kp.data.dao.OrderDao
import com.example.ia4_bakery_kp.data.dao.UserDao
import com.example.ia4_bakery_kp.data.entity.Cake
import com.example.ia4_bakery_kp.data.entity.Order
import com.example.ia4_bakery_kp.data.entity.OrderCakeCrossRef
import com.example.ia4_bakery_kp.data.entity.User

// Singleton Room database:

@Database(entities = [User::class, Cake::class, Order::class,
OrderCakeCrossRef::class], version = 5)
// Room database with 4 tables/entities, version 5, using converters to
handle date and time conversions
@TypeConverters(Converters::class) // Converts 'Date' to 'Long' when
saving (writing) to the database & 'Long' back to 'Date' when reading
from the database
// As the room database cannot store 'Date'
abstract class AppDatabase : RoomDatabase() {
    // Abstract DAO getters to access DAOs for querying
    // Room generates the implementations that run the SQL which is
defined in the DAOs
    abstract fun userDao(): UserDao
    abstract fun cakeDao(): CakeDao
    abstract fun orderDao(): OrderDao

    // Singleton instance of the database (only 1 instance of the
database exists for the app)
    // The database is stored in 'INSTANCE' to ensure every different
part of the app all use the same database
    companion object {
        @Volatile // '@Volatile' makes all threads see changes to
INSTANCE immediately / see the latest 'INSTANCE' value
        // This (as well as 'Synchronized') prevents race conditions
where 2 threads access INSTANCE at the same time
        private var INSTANCE: AppDatabase? = null
```

```
        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) { // If 'INSTANCE' is
not null return it
                // Otherwise allow only 1 thread to run this block at a
time, to ensure only 1 instance of the database is created
                // Build the database, save it to 'INSTANCE' and return
it
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "bakery_database"
                ).fallbackToDestructiveMigration().build() //
Automatically wipes the database if the schema version changes
                INSTANCE = instance // save the instance in 'INSTANCE'
                instance // return the instance of the database
            }
        }
    }
}
```

Converters.kt:

```
package com.example.ia4_bakery_kp.data

import androidx.room.TypeConverter
import java.util.Date

class Converters {
    @TypeConverter // Convert from a Long to a Date:
    fun fromTimestamp(value: Long?): Date? {
        return value?.let { Date(it) }
    }

    @TypeConverter // Convert from a Date to a Long:
    fun dateToTimestamp(date: Date?): Long? {
        return date?.time
    }
}
```

4.5. Navigation

NavGraph.kt:

```
package com.example.ia4_bakery_kp.navigation

import android.app.Application
import androidx.compose.foundation.layout.padding
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Home
import androidx.compose.material.icons.filled.Menu
import androidx.compose.material3.CenterAlignedTopAppBar
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
```

```
import androidx.compose.material3.NavigationBar
import androidx.compose.material3.NavigationBarItem
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.remember
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavDestination.Companion.hierarchy
import androidx.navigation.NavGraph.Companion.findStartDestination
import androidx.navigation.NavHostController
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.currentBackStackEntryAsState
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.example.ia4_bakery_kp.R
import com.example.ia4_bakery_kp.data.AppDatabase
import com.example.ia4_bakery_kp.data.entity.Cake
import com.example.ia4_bakery_kp.data.repository.CakeRepository
import com.example.ia4_bakery_kp.data.repository.UserRepository
import com.example.ia4_bakery_kp.ui.home.HomeScreen
import com.example.ia4_bakery_kp.ui.login.LoginForm
import com.example.ia4_bakery_kp.ui.menu.CakeMenuScreen
import com.example.ia4_bakery_kp.ui.order.OrderReceiptScreen
import com.example.ia4_bakery_kp.ui.order.ShoppingCartScreen
import com.example.ia4_bakery_kp.ui.signup.SignUpForm
import com.example.ia4_bakery_kp.viewmodel.CakeViewModel
import com.example.ia4_bakery_kp.viewmodel.CakeViewModelFactory
import com.example.ia4_bakery_kp.viewmodel.OrderViewModel
import com.example.ia4_bakery_kp.viewmodel.OrderViewModelFactory
import com.example.ia4_bakery_kp.viewmodel.UserViewModel
import com.example.ia4_bakery_kp.viewmodel.UserViewModelFactory

// Screen definitions:
sealed class Screen(val route: String, val label: String? = null, val
icon: @Composable (() -> Unit)? = null) {
    object SignUp : Screen("signup")
    object Login : Screen("login")
    object Home : Screen("home", "Home", { Icon(Icons.Filled.Home,
null) })
    object Menu : Screen("menu", "Cake Menu", { Icon(Icons.Filled.Menu,
null) })
    object ShoppingCart : Screen("shopping_cart")
    object OrderReceipt : Screen("order_receipt")
}

// List of bottom navigation items:
val bottomNavItems = listOf(
    Screen.Home,
    Screen.Menu,
)
```

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun NavGraph() {
    val navController = rememberNavController()
    val context = LocalContext.current
    val application = context.applicationContext as Application

    // Initialize the 'ViewModels' using factories
    val db = remember { AppDatabase.getDatabase(context) }
    val cakeRepository = remember { CakeRepository(db.cakeDao()) }
    val userRepository = remember { UserRepository(db.userDao()) }

    val cakeViewModel: CakeViewModel = viewModel(factory =
CakeViewModelFactory(cakeRepository))
    val userViewModel: UserViewModel = viewModel(factory =
UserViewModelFactory(userRepository))
    val orderViewModel: OrderViewModel = viewModel(factory =
OrderViewModelFactory(application))

    // Add the desserts to the database once (if database is empty):
    LaunchedEffect(Unit) {
        if (cakeRepository.getAllCakes().isEmpty()) {
            cakeRepository.insertCake(
                Cake(name = "Decadent Chocolate Fudge Cake (Slice)",
price = 5.00, description = "Rich milk chocolate delight", imageResId =
R.drawable.chocolate_cake)
            )
            cakeRepository.insertCake(
                Cake(name = "Classic Vanilla Bean Cake (Slice)", price
= 5.50, description = "A fluffy, moist vanilla sponge infused with
vanilla beans, layered with a smooth buttercream frosting. A timeless
favorite perfect for any occasion", imageResId =
R.drawable.vanilla_cake)
            )
            cakeRepository.insertCake(
                Cake(name = "Fresh Strawberry Cream Cake (Slice)",
price = 6.00, description = "Light strawberry-infused sponge cake
layered with fresh strawberry preserves and whipped cream cheese
frosting", imageResId = R.drawable.strawberry_cake)
            )
            cakeRepository.insertCake(
                Cake(name = "Classic Mini Cinnamon Rolls (3pk)", price
= 5.50, description = "Warm dough swirled with cinnamon sugar and
topped with cream cheese icing", imageResId =
R.drawable.cinnamon_rolls)
            )
            cakeRepository.insertCake(
                Cake(name = "Classic New York Style Cheesecake
(Slice)", price = 8.00, description = "Rich and creamy cheesecake on a
buttery graham cracker crust", imageResId = R.drawable.cheesecake)
            )
            cakeRepository.insertCake(
                Cake(name = "Chocolate Fudge Brownie", price = 4.00,
description = "Dense, dark chocolate fudge brownie topped with crunchy
walnuts", imageResId = R.drawable.brownie)
            )
            cakeRepository.insertCake(
```

```
                Cake(name = "Chocolate Chunk Cookie", price = 3.50,
description = "Giant, chewy cookie loaded with semi-sweet chocolate
chunks", imageResId = R.drawable.cookie)
            )
            cakeRepository.insertCake(
                Cake(name = "Traditional Chocolate Eclair", price =
5.00, description = "A crisp choux pastry shell filled with cool
vanilla pastry cream and dipped in a glossy, semi-sweet chocolate
glaze", imageResId = R.drawable.eclair)
            )
            cakeRepository.insertCake(
                Cake(name = "Assorted Macarons (3pk)", price = 10.00,
description = "Delicate almond meringue cookies with ganache fillings",
imageResId = R.drawable.macarons)
            )
            cakeRepository.insertCake(
                Cake(name = "Classic Apple Pie", price = 6.50,
description = "Flaky pastry filled with sweet, spiced apples",
imageResId = R.drawable.apple_pie)
            )
        }
    }

    // Determine if the bottom bar should be shown
    val navBackStackEntry by
navController.currentBackStackEntryAsState()
    val currentDestination = navBackStackEntry?.destination
    val showBottomBar = bottomNavItems.any { it.route ==
currentDestination?.route?.substringBefore("/") }

    Scaffold(
        topBar = { CenterAlignedTopAppBar(title = { Text("Elite Bake
Shop App") }) },
        bottomBar = { if (showBottomBar) {
BottomNavigationBar(navController) } }
    ) { innerPadding ->
        // Navigation graph:
        // Starting screen is the signup page
        NavHost(navController, startDestination = Screen.SignUp.route,
Modifier.padding(innerPadding)) {
            composable(Screen.SignUp.route) {
                SignUpForm(viewModel = cakeViewModel, userViewModel =
userViewModel, navController = navController)
            }
            composable(Screen.Login.route) {
                LoginForm(userViewModel = userViewModel, navController
= navController)
            }
            composable(Screen.Home.route) {
                HomeScreen(
                    userViewModel = userViewModel,
                    email = userViewModel.loggedInEmail ?: ""
                )
            }
            composable(Screen.Menu.route) {
                CakeMenuScreen(cakeViewModel, orderViewModel) {
navController.navigate(Screen.ShoppingCart.route) }
```

```
            }
            composable(Screen.ShoppingCart.route) {
                ShoppingCartScreen(
                    viewModel = orderViewModel,
                    onConfirmOrderClick = {

navController.navigate(Screen.OrderReceipt.route) {
                            popUpTo(Screen.Menu.route)
                        }
                    },
                    onBackToMenuClick = {
                        navController.navigate(Screen.Menu.route)
                    }
                )
            }
            composable(Screen.OrderReceipt.route) {
                OrderReceiptScreen(viewModel = orderViewModel) {
                    navController.popBackStack(Screen.Menu.route,
inclusive = false)
                }
            }
        }
    }
}

// Bottom navigation bar:
@Composable
fun BottomNavigationBar(navController: NavHostController) {
    NavigationBar {
        val navBackStackEntry by
navController.currentBackStackEntryAsState()
        val currentDestination = navBackStackEntry?.destination

        bottomNavItems.forEach { screen ->
            NavigationBarItem(
                icon = { screen.icon?.invoke()!! },
                label = { Text(screen.label!!) },
                selected = currentDestination?.hierarchy?.any {
it.route?.substringBefore("/") == screen.route } == true,
                onClick = {
                    val route = screen.route
                    navController.navigate(route) {

popUpTo(navController.graph.findStartDestination().id) { saveState =
true }
                        launchSingleTop = true
                        restoreState = true
                    }
                }
            )
        }
    }
}
```

4.6. UI

HomeScreen.kt:

```
package com.example.ia4_bakery_kp.ui.home

import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import com.example.ia4_bakery_kp.R
import com.example.ia4_bakery_kp.viewmodel.UserViewModel

@Composable
fun HomeScreen(userViewModel: UserViewModel, email: String) {
    Surface(
        modifier = Modifier.fillMaxSize(),
        color = Color(0xFFFFF0F5)
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(24.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Text(
                "Elite Bake Shop",
                style = MaterialTheme.typography.headlineLarge,
                color = Color(0xFF8B4513),
                textAlign = TextAlign.Center
            )
            // Logo for the fake bake shop brand
            Spacer(Modifier.height(8.dp))
            Image(
                painter = painterResource(id = R.drawable.cake_home),
                contentDescription = "Home Screen Cake",
                modifier = Modifier
                    .height(160.dp)
                    .clip(CircleShape)
            )
```

```
            Spacer(Modifier.height(24.dp))

            // Personalized greeting, uses the users email
            Text(
                "Welcome back, $email!",
                style = MaterialTheme.typography.bodyLarge,
                textAlign = TextAlign.Center
            )

            // Generic brand mission statement for the home screen
            Spacer(Modifier.height(12.dp))
            Text(
                "At Elite Bake Shop, we believe every day deserves a
sweet moment. Forget the ordinary, we specialize in crafting decadent
cakes, flaky pastries, and comforting classics, all made with the
finest ingredients and a whole lot of love.\n" +
                        "\n" +
                        "What makes us special?\n" +
                        "\n" +
                        "Handcrafted Quality: From the rich, dark fudge
in our brownies to the creamy filling in our éclairs, everything is
baked fresh daily by passionate pastry chefs.\n" +
                        "\n" +
                        "Ready to find your next favorite dessert?",
                style = MaterialTheme.typography.bodyMedium,
                textAlign = TextAlign.Center
            )
        }
    }
}
```

LogIn.kt:

```
package com.example.ia4_bakery_kp.ui.login

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.navigation.NavController
import androidx.navigation.NavGraph.Companion.findStartDestination
import com.example.ia4_bakery_kp.navigation.Screen
import com.example.ia4_bakery_kp.viewmodel.UserViewModel

@Composable
fun LoginForm(
    userViewModel: UserViewModel,
    navController: NavController,
) {
    var email by remember { mutableStateOf("") }
    var errorMessage by remember { mutableStateOf("") }

    Column(
```

```
        modifier = Modifier.fillMaxSize().padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text("Log In", style = MaterialTheme.typography.headlineMedium)

        // Email input field:
        TextField(value = email, onValueChange = { email = it }, label
= { Text("Email") })

        Spacer(Modifier.height(16.dp))
        Button(onClick = {
            if (email.isBlank()) { // Check if email is empty
                errorMessage = "Please enter your email"
            } else {
                userViewModel.loginUser(email) { user ->
                    if (user != null) { // If the user exists (email
was registered in sign up), navigate to the home screen
                        // This allows new users to log in as well as
existing users
                        // Not fully functional however, as sign up
page is always shown no matter what
                        navController.navigate(Screen.Home.route) {

popUpTo(navController.graph.findStartDestination().id) {
                                inclusive = true
                            }
                        }
                    } else { // If the user does not exist, display an
error message
                        errorMessage = "Invalid email"
                    }
                }
            }
        }) {
            Text("Log In")
        }

        // Display error message (if there is an error)
        if (errorMessage.isNotEmpty()) {
            Text(errorMessage, color = MaterialTheme.colorScheme.error)
        }
    }
}
```

CakeMenuScreen.kt:

```
package com.example.ia4_bakery_kp.ui.menu

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ShoppingCart
import androidx.compose.material3.*
```

```
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import com.example.ia4_bakery_kp.viewmodel.CakeViewModel
import com.example.ia4_bakery_kp.viewmodel.OrderViewModel
import kotlinx.coroutines.launch

@Composable
fun CakeMenuScreen(
    cakeViewModel: CakeViewModel,
    orderViewModel: OrderViewModel,
    onGoToCartClick: () -> Unit
) {
    // Observe cakes from ViewModel
    val cakes by cakeViewModel.cakes.collectAsState()

    // Local state for search query
    var searchQuery by remember { mutableStateOf("") }

    // Load cakes when screen opens
    LaunchedEffect(Unit) {
        cakeViewModel.loadCakes()
    }

    // Snackbar & coroutine scope for feedback
    val snackbarHostState = remember { SnackbarHostState() }
    val scope = rememberCoroutineScope()

    // Observe cart count for badge
    val cartCount =
orderViewModel.cartState.collectAsState().value.values.sum()

    Scaffold(
        modifier = Modifier.fillMaxSize(),
        containerColor = Color(0xFFFFF0F5),
        snackbarHost = { SnackbarHost(snackbarHostState) }, // host for
SnackBars
        floatingActionButton = {
            // BadgedBox wraps the cart icon to show item count
            BadgedBox(
                badge = {
                    if (cartCount > 0) {
                        Badge { Text(cartCount.toString()) }
                    }
                }
            ) {
                FloatingActionButton(onClick = onGoToCartClick) {
                    Icon(Icons.Filled.ShoppingCart, contentDescription
= "Shopping Cart")
                }
            }
        }
    ) { paddingValues ->
        Column(Modifier.padding(paddingValues).padding(12.dp)) {
            Text("Cake Menu", style =
```

```
MaterialTheme.typography.headlineSmall)
            Spacer(Modifier.height(8.dp))

            // Cake Menu search bar:
            OutlinedTextField(
                value = searchQuery,
                onValueChange = {
                    searchQuery = it
                    cakeViewModel.searchCakes(it) // call ViewModel
search
                },
                label = { Text("Search treats") },
                singleLine = true,
                modifier = Modifier.fillMaxWidth()
            )

            Spacer(Modifier.height(12.dp))

            // Desserts list:
            // Displays all treat types dynamically from database
            // 'LazyColum' allows you to scroll through the list of
desserts
            LazyColumn {
                items(cakes) { cake ->
                    Card(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(vertical = 8.dp)
                    ) {
                        Row(
                            modifier = Modifier.padding(12.dp),
                            verticalAlignment =
Alignment.CenterVertically
                        ) {
                            Column(modifier = Modifier.weight(1f)) {
                                Text(cake.name, style =
MaterialTheme.typography.titleMedium)
                                Text(
                                    "$${"%.2f".format(cake.price)}",
                                    style =
MaterialTheme.typography.bodyMedium
                                )
                                Text(cake.description, style =
MaterialTheme.typography.bodySmall)
                            }

                            // Add to Cart button with feedback for
each dessert
                            Button(onClick = {
                                orderViewModel.addToCart(cake)
                                // Show SnackBar confirmation message
of the item added to the cart
                                // Without this it is hard to tell if
anything has been added to the cart as a user
                                scope.launch {

snackbarHostState.showSnackbar("${cake.name} added to cart")
```

```
                                    }
                                }) {
                                    Text("Add to Cart")
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}
```

OrderScreen.kt:

```
package com.example.ia4_bakery_kp.ui.order

import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import com.example.ia4_bakery_kp.viewmodel.CakeViewModel
import com.example.ia4_bakery_kp.R

@Composable
fun OrderScreen(viewModel: CakeViewModel) {
    var quantityText by rememberSaveable { mutableStateOf("1") }
    val confirmedMessage by
viewModel.confirmationMessage.collectAsState()
    val quantity by viewModel.quantity.collectAsState()

    // Pull selected cake from ViewModel or fallback
```

```
    val cake = viewModel.selectedCake ?:
com.example.ia4_bakery_kp.data.entity.Cake(
        id = 1,
        name = "Chocolate Cake",
        price = 10.0,
        description = "Rich chocolate delight",
        imageResId = R.drawable.chocolate_cake
    )

    Surface(
        modifier = Modifier.fillMaxSize(),
        color = Color(0xFFFFF0F5)
    ) {
        Column(
            Modifier.padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text("Order: ${cake.name}", style =
MaterialTheme.typography.headlineSmall)
            Spacer(Modifier.height(12.dp))
            // Show image of treat
            Image(
                painter = painterResource(id = cake.imageResId),
                contentDescription = cake.name,
                modifier = Modifier
                    .height(160.dp)
                    .clip(CircleShape)
            )
            Spacer(Modifier.height(12.dp))
            Text("Price: $${"%.2f".format(cake.price)}", style =
MaterialTheme.typography.bodyLarge)
            Spacer(Modifier.height(8.dp))

            OutlinedTextField( // Styled text field for quantity input
                value = quantityText,
                onValueChange = { newText -> // Only update
quantityText if the user input is a number
                    if (newText.all { it.isDigit() }) {
                        quantityText = newText
                    }
                },
                label = { Text("Order Quantity") },
                keyboardOptions = KeyboardOptions(
                    keyboardType = KeyboardType.Number,
                    imeAction = ImeAction.Done
                ),
                singleLine = true,
                modifier = Modifier.fillMaxWidth(0.6f)
            )

            Spacer(Modifier.height(12.dp))

            Button(onClick = {
                val q = quantityText.toIntOrNull() ?: 0 // Try to
convert to int if that fails, default to 0

                if (q == 0) {
```

```
                    viewModel.setConfirmationMessage("Warning: Can't
order 0 cakes") // Display an warning message if the quantity is 0
                } else {
                    viewModel.setQuantity(q)
                    viewModel.placeOrder() // Call 'placeOrder'
function to display confirmation message
                }
            }, modifier = Modifier.fillMaxWidth(0.25f)) {
                Text("Order")
            }

            Spacer(Modifier.height(12.dp))
            if (confirmedMessage.isNotBlank()) {
                Text(confirmedMessage, style =
MaterialTheme.typography.bodyLarge)
            }
        }
    }
}
```

ShoppingCartScreen.kt:

```
package com.example.ia4_bakery_kp.ui.order

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.Remove
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import com.example.ia4_bakery_kp.data.entity.Cake
import com.example.ia4_bakery_kp.viewmodel.OrderViewModel
import com.example.ia4_bakery_kp.navigation.Screen
import androidx.navigation.NavController

@Composable
fun ShoppingCartScreen(
    viewModel: OrderViewModel,
    onConfirmOrderClick: () -> Unit,
    onBackToMenuClick: () -> Unit
) {
    // Observe the cart & total price from ViewModel
    val cart by viewModel.cartState.collectAsState()
```

```
    val totalPrice by viewModel.totalPrice.collectAsState()

    Surface(
        modifier = Modifier.fillMaxSize(),
        color = Color(0xFFFFF0F5)
    ) {
        Column(modifier = Modifier.padding(12.dp)) {
            Text("Your Cart", style =
MaterialTheme.typography.headlineSmall)
            Spacer(Modifier.height(12.dp))

            // If the cart is empty inform the user and provide them
with a button to go back to the menu
            if (cart.isEmpty()) {
                Text("Your cart is empty.", style =
MaterialTheme.typography.bodyMedium)
                Spacer(Modifier.height(16.dp))
                Button(
                    onClick = { onBackToMenuClick() },
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Text("Back to Menu")
                }
            } else {
                // Otherwise display the desserts they added to the
cart as a 'LazyColumn' so they can scroll the list if it is long
                LazyColumn(modifier = Modifier.weight(1f)) {
                    items(cart.toList()) { (cake, quantity) ->
                        CartItemRow(cake, quantity, viewModel)
                    }
                }

                Spacer(Modifier.height(16.dp))

                // Display the total price of all the items in the
cart, adjust with quantity changes and item removal
                Text(
                    "Total: $${"%.2f".format(totalPrice)}",
                    style = MaterialTheme.typography.headlineSmall,
                    modifier = Modifier.align(Alignment.End)
                )

                Spacer(Modifier.height(16.dp))

                Button(
                    onClick = {
                        viewModel.confirmOrder()
                        onConfirmOrderClick()
                    },
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Text("Confirm Order")
                }
            }
        }
    }
}
```

```
@Composable
fun CartItemRow(cake: Cake, quantity: Int, viewModel: OrderViewModel) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 8.dp),
        verticalAlignment = Alignment.CenterVertically
    ) { // Display the name and price of the desserts in the cart
        Column(modifier = Modifier.weight(1f)) {
            Text(cake.name, style = MaterialTheme.typography.bodyLarge)
            Text("$${"%.2f".format(cake.price)}", style =
MaterialTheme.typography.bodyMedium)
        }

        // Button to decrease the quantity of an item in the cart
        Row(verticalAlignment = Alignment.CenterVertically) {
            IconButton(onClick = { viewModel.decreaseCartItem(cake) })
{
                Icon(Icons.Filled.Remove, contentDescription =
"Decrease quantity")
            }
            Text(
                "$quantity",
                modifier = Modifier.padding(horizontal = 8.dp),
                textAlign = TextAlign.Center
            )
            // Button to increase the quantity of an item in the cart
            IconButton(onClick = { viewModel.addToCart(cake) }) {
                Icon(Icons.Filled.Add, contentDescription = "Increase
quantity")
            }
            // Button to remove an item from the cart
            IconButton(onClick = { viewModel.removeFromCart(cake) }) {
                Icon(Icons.Filled.Delete, contentDescription = "Remove
from cart")
            }
        }
    }
}
```

OrderReceiptScreen.kt:

```
package com.example.ia4_bakery_kp.ui.order

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
```

```
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import com.example.ia4_bakery_kp.viewmodel.OrderViewModel
import java.text.SimpleDateFormat
import java.util.Locale

@Composable
fun OrderReceiptScreen(viewModel: OrderViewModel, onBackToMenuClick: ()
-> Unit) {
    val confirmedOrder by viewModel.confirmedOrder.collectAsState()
    val dateFormatter = SimpleDateFormat("MM/dd/yyyy hh:mm a",
Locale.getDefault()) // Format date for the order receipt

    Surface(
        modifier = Modifier.fillMaxSize(),
        color = Color(0xFFFFF0F5)
    ) {
        // Inform the user of a successful order:
        Column(modifier = Modifier.padding(12.dp)) {
            Text("Order Confirmed!", style =
MaterialTheme.typography.headlineSmall)
            Spacer(Modifier.height(16.dp))

            confirmedOrder?.let { orderWithCakes ->
                // Display the 'orderId', 'orderDate', 'items', and
'total' for the order receipt
                Text("Order ID: ${orderWithCakes.order.orderId}", style
= MaterialTheme.typography.bodyLarge)
                Text("Date:
${dateFormatter.format(orderWithCakes.order.orderDate)}", style =
MaterialTheme.typography.bodyMedium)
                Spacer(Modifier.height(12.dp))

                Text("Items:", style =
MaterialTheme.typography.titleMedium)
                orderWithCakes.cakes.forEach {
                    Text("- ${it.name}")
                }

                Spacer(Modifier.height(12.dp))
                Text("Total:
$${"%.2f".format(orderWithCakes.order.totalPrice)}", style =
MaterialTheme.typography.headlineSmall)
            } ?: run {
                Text("Loading order details...") // Display a loading
message if the order is takes a long time to load
            }

            // Back to Menu button
            Spacer(Modifier.weight(1f))
            Button(onClick = onBackToMenuClick, modifier =
Modifier.fillMaxWidth()) {
                Text("Back to Menu")
```

```
            }
        }
    }
}
```

SignUp.kt:

```
package com.example.ia4_bakery_kp.ui.signup

import android.util.Patterns
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.selection.selectable
import androidx.compose.foundation.selection.selectableGroup
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Email
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.Phone
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.semantics.Role
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.navigation.NavController
import com.example.ia4_bakery_kp.navigation.Screen
import com.example.ia4_bakery_kp.data.entity.User
import com.example.ia4_bakery_kp.viewmodel.CakeViewModel
import com.example.ia4_bakery_kp.viewmodel.UserViewModel

@Composable
fun SignUpForm(
    viewModel: CakeViewModel,
    userViewModel: UserViewModel,
    navController: NavController
) {
    var name by remember { mutableStateOf("") }
    var email by remember { mutableStateOf("") }
    var phoneNumber by remember { mutableStateOf("") }
    var selectedGender by remember { mutableStateOf("") }
    var selectedPayment by remember { mutableStateOf("") }
    var isAgreed by remember { mutableStateOf(false) }

    // Validation error states
    var nameError by remember { mutableStateOf<String?>(null) }
    var emailError by remember { mutableStateOf<String?>(null) }
    var phoneError by remember { mutableStateOf<String?>(null) }
    var genderError by remember { mutableStateOf<String?>(null) }
    var paymentError by remember { mutableStateOf<String?>(null) }
    var termsError by remember { mutableStateOf<String?>(null) }
```

```
    var showPopUp by remember { mutableStateOf(false) } // Initially
False

    fun validate() {
        nameError = if (name.isBlank()) "Please enter your name." else
null
        emailError = if
(!Patterns.EMAIL_ADDRESS.matcher(email).matches()) "Invalid email" else
null
        phoneError = when {
            phoneNumber.isBlank() -> "Phone number cannot be empty"
            !phoneNumber.all { it.isDigit() } -> "Phone number must be
numeric"
            phoneNumber.length < 10 -> "Phone number must be at least
10 digits"
            else -> null
        }
        genderError = if (selectedGender.isBlank()) "Please select your
gender" else null
        paymentError = if (selectedPayment.isBlank()) "Please select a
payment method" else null
        termsError = if (!isAgreed) "You must agree to the terms" else
null
    }

    Box(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .verticalScroll(rememberScrollState()),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Top
        ) {
            Spacer(modifier = Modifier.height(40.dp))
            Text(
                "Elite Bakery App ",
                modifier = Modifier.padding(4.dp),
                style =
MaterialTheme.typography.titleLarge.copy(fontWeight = FontWeight.Bold)
            )
            Text("Sign Up:",
                fontWeight = FontWeight.Bold,
                modifier = Modifier.padding(4.dp)
            )

            // Full Name input field
            TextField(
                value = name,
                onValueChange = { name = it; validate() },
                label = { Text("Full Name:") },
                leadingIcon = { Icon(imageVector = Icons.Filled.Person,
contentDescription = "Name Icon") },
                modifier = Modifier.fillMaxWidth(),
```

```
                isError = nameError != null
            )
            nameError?.let { Text(text = it, color =
MaterialTheme.colorScheme.error) }

            Spacer(modifier = Modifier.height(16.dp))

            // Email input field
            TextField(
                value = email,
                onValueChange = { email = it; validate() },
                label = { Text("Email Address:") },
                modifier = Modifier.fillMaxWidth(),
                leadingIcon = { Icon(imageVector = Icons.Filled.Email,
contentDescription = "Email Icon") },
                keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Email),
                isError = emailError != null
            )
            emailError?.let { Text(text = it, color =
MaterialTheme.colorScheme.error) }

            Spacer(modifier = Modifier.height(16.dp))

            // Phone number input field
            TextField(
                value = phoneNumber,
                onValueChange = { phoneNumber = it; validate() },
                label = { Text("Phone Number: no symbols or spaces") },
                modifier = Modifier.fillMaxWidth(),
                leadingIcon = { Icon(imageVector = Icons.Filled.Phone,
contentDescription = "Phone Icon") },
                keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Number),
                isError = phoneError != null
            )
            phoneError?.let { Text(text = it, color =
MaterialTheme.colorScheme.error) }

            Spacer(modifier = Modifier.height(36.dp))

            // Gender selection
            Text("Gender:", fontWeight = FontWeight.Bold, modifier =
Modifier.fillMaxWidth().padding(start = 16.dp))
            val genderOptions = listOf("Male", "Female", "Other")
            Column(modifier = Modifier.selectableGroup()) {
                genderOptions.forEach { gender ->
                    Row(
                        modifier =
Modifier.fillMaxWidth().height(36.dp).selectable(
                            selected = (gender == selectedGender),
                            onClick = { selectedGender = gender;
validate() },
                            role = Role.RadioButton
                        ).padding(horizontal = 16.dp),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
```

```
                        RadioButton(selected = (gender ==
selectedGender), onClick = null)
                        Text(text = gender, style =
MaterialTheme.typography.bodyLarge, modifier = Modifier.padding(start =
16.dp))
                    }
                }
            }
            genderError?.let { Text(text = it, color =
MaterialTheme.colorScheme.error) }

            Spacer(modifier = Modifier.height(26.dp))

            // Payment method selection
            Text("Payment Method:", fontWeight = FontWeight.Bold,
modifier = Modifier.fillMaxWidth().padding(start = 16.dp))
            val paymentOptions = listOf("Direct Deposit", "ePayment")
            Column(modifier = Modifier.selectableGroup()) {
                paymentOptions.forEach { payment ->
                    Row(
                        modifier =
Modifier.fillMaxWidth().height(36.dp).selectable(
                            selected = (payment == selectedPayment),
                            onClick = { selectedPayment = payment;
validate() },
                            role = Role.RadioButton
                        ).padding(horizontal = 16.dp),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        RadioButton(selected = (payment ==
selectedPayment), onClick = null)
                        Text(text = payment, style =
MaterialTheme.typography.bodyLarge, modifier = Modifier.padding(start =
16.dp))
                    }
                }
            }
            paymentError?.let { Text(text = it, color =
MaterialTheme.colorScheme.error) }

            Spacer(modifier = Modifier.height(26.dp))

            // Terms agreement switch
            Row(verticalAlignment = Alignment.CenterVertically) {
                Switch(checked = isAgreed, onCheckedChange = { isAgreed
= it; validate() })
                Text(" Agree to Terms")
            }
            termsError?.let { Text(text = it, color =
MaterialTheme.colorScheme.error) }

            Spacer(modifier = Modifier.height(36.dp))

            // Sign-up Button with form validation
            Button(onClick = {
                validate()
                if (nameError == null && emailError == null &&
```

```
phoneError == null && genderError == null && paymentError == null &&
termsError == null) {
                    val newUser = User(
                        email = email,
                        name = name,
                        phone = phoneNumber,
                        gender = selectedGender,
                        paymentMethod = selectedPayment
                    )
                    userViewModel.registerUser(newUser)
                    navController.navigate(Screen.Login.route)
                } else {
                    showPopUp = true
                }
            }) {
                Text("Sign Up")
            }

            // Alert Dialog pop up for incomplete form
            if (showPopUp) {
                AlertDialog(
                    onDismissRequest = { showPopUp = false },
                    title = { Text("Incomplete Form") },
                    text = { Text("Please fill all the values!") },
                    confirmButton = {
                        Button(onClick = { showPopUp = false }) {
Text("OK") }
                    }
                )
            }
        }
    }
}
```

4.7. ViewModel

CakeViewModel.kt:

```
package com.example.ia4_bakery_kp.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.ia4_bakery_kp.data.entity.Cake
import com.example.ia4_bakery_kp.data.repository.CakeRepository
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

class CakeViewModel(private val repository: CakeRepository) :
ViewModel() {

    // Holds the list of cakes for the menu
    private val _cakes = MutableStateFlow<List<Cake>>(emptyList())
    val cakes: StateFlow<List<Cake>> = _cakes.asStateFlow()
```

```
    // Holds the selected cake
    var selectedCake: Cake? = null
        private set

    // Quantity and confirmation message
    private val _quantity = MutableStateFlow(1)
    val quantity: StateFlow<Int> = _quantity.asStateFlow()

    private val _confirmationMessage = MutableStateFlow("")
    val confirmationMessage: StateFlow<String> =
_confirmationMessage.asStateFlow()

    // Load all cakes from the database:
    fun loadCakes() {
        viewModelScope.launch {
            _cakes.value = repository.getAllCakes()
        }
    }

    // Search cakes by name, not case-sensitive:
    fun searchCakes(query: String) {
        viewModelScope.launch {
            _cakes.value = if (query.isBlank()) {
                repository.getAllCakes()
            } else {
                repository.searchCakes(query)
            }
        }
    }

    // Set the quantity of the selected dessert:
    fun setQuantity(q: Int) {
        _quantity.value = if (q >= 0) q else 0
    }

    // Set the confirmation message:
    fun setConfirmationMessage(message: String) {
        _confirmationMessage.value = message
    }

    // Place an order:
    fun placeOrder() {
        val price = selectedCake?.price ?: 10.0
        val total = price * _quantity.value
        _confirmationMessage.value =
            "Order successfully placed - Total:
$${"%.2f".format(total)}"
    }
}
```

CakeViewModelFactory.kt:

```
package com.example.ia4_bakery_kp.viewmodel

import androidx.lifecycle.ViewModel
```

```
import androidx.lifecycle.ViewModelProvider
import com.example.ia4_bakery_kp.data.repository.CakeRepository

// Factory class for CakeViewModel: builds and returns a CakeViewModel
when requested by ViewModelProvider
class CakeViewModelFactory(private val repository: CakeRepository) :
ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        if (modelClass.isAssignableFrom(CakeViewModel::class.java)) {
// Check if the requested ViewModel is of type CakeViewModel
            @Suppress("UNCHECKED_CAST")
            return CakeViewModel(repository) as T // Create and return
a new instance of CakeViewModel
        }
        throw IllegalArgumentException("Unknown ViewModel class") //
Throw an exception if the requested ViewModel is not of the expected
type
    }
}
```

OrderViewModel.kt:

```
package com.example.ia4_bakery_kp.viewmodel

import android.app.Application
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.viewModelScope
import com.example.ia4_bakery_kp.data.AppDatabase
import com.example.ia4_bakery_kp.data.entity.Cake
import com.example.ia4_bakery_kp.data.entity.Order
import com.example.ia4_bakery_kp.data.entity.OrderWithCakes
import com.example.ia4_bakery_kp.data.repository.OrderRepository
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.map
import kotlinx.coroutines.flow.stateIn
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch
import java.util.Date

class OrderViewModel(application: Application) : ViewModel() {

    private val orderRepository: OrderRepository

    // Holds the current shopping cart state <Cake, Quantity>
    private val _cartState = MutableStateFlow<Map<Cake,
Int>>(emptyMap())
    val cartState: StateFlow<Map<Cake, Int>> = _cartState.asStateFlow()
```

```
    // Holds the most recently confirmed order details for the receipt
screen
    private val _confirmedOrder =
MutableStateFlow<OrderWithCakes?>(null)
    val confirmedOrder: StateFlow<OrderWithCakes?> =
_confirmedOrder.asStateFlow()

    // Reactively calculate the total price whenever the cart changes
    val totalPrice: StateFlow<Float> = cartState.map { cart ->
        cart.map { (cake, quantity) -> cake.price.toFloat() * quantity
}.sum()
    }.stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5000),
        initialValue = 0f
    )

    init {
        val orderDao = AppDatabase.getDatabase(application).orderDao()
        orderRepository = OrderRepository(orderDao)
    }

    // Add an item to the cart and update the total price:
    fun addToCart(cake: Cake) {
        _cartState.update {
            val currentCart = it.toMutableMap()
            val currentQuantity = currentCart.getOrDefault(cake, 0)
            currentCart[cake] = currentQuantity + 1
            currentCart
        }
    }

    // Decrease the quantity of an item in the cart:
    fun decreaseCartItem(cake: Cake) {
        _cartState.update {
            val currentCart = it.toMutableMap()
            val currentQuantity = currentCart.getOrDefault(cake, 0)
            if (currentQuantity > 1) {
                currentCart[cake] = currentQuantity - 1
            } else {
                currentCart.remove(cake)
            }
            currentCart
        }
    }

    // Remove an item from the cart:
    fun removeFromCart(cake: Cake) {
        _cartState.update {
            val currentCart = it.toMutableMap()
            currentCart.remove(cake)
            currentCart
        }
    }

    // Clear the cart:
```

```
    private fun clearCart() {
        _cartState.value = emptyMap()
    }

    // Confirm the order:
    fun confirmOrder() {
        viewModelScope.launch {
            val cart = cartState.value
            if (cart.isNotEmpty()) { // Only confirm if the cart is not
empty
                val newOrder = Order(
                    totalPrice = totalPrice.value,
                    orderDate = Date()
                )
                val cakeIds = cart.keys.map { it.id }

                val orderId = orderRepository.addOrder(newOrder,
cakeIds)
                _confirmedOrder.value =
orderRepository.getOrderWithCakes(orderId.toInt())

                // Clear the cart after order is confirmed
                clearCart()
            }
        }
    }
}

// Factory class for OrderViewModel: builds and returns a
OrderViewModel when requested by ViewModelProvider
class OrderViewModelFactory(private val application: Application) :
ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        if (modelClass.isAssignableFrom(OrderViewModel::class.java)) {
// Check if the requested ViewModel is of type OrderViewModel
            @Suppress("UNCHECKED_CAST")
            return OrderViewModel(application) as T // Create and
return a new instance of OrderViewModel
        }
        throw IllegalArgumentException("Unknown ViewModel class") //
Throw an exception if the requested ViewModel is not of the expected
type
    }
}
```

UserViewModel.kt:

```
package com.example.ia4_bakery_kp.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.ia4_bakery_kp.data.entity.User
import com.example.ia4_bakery_kp.data.repository.UserRepository
import kotlinx.coroutines.launch
```

```
class UserViewModel(private val repository: UserRepository) :
ViewModel() {

    var loggedInEmail: String? = null
        private set

    // Register a new user:
    fun registerUser(user: User) {
        viewModelScope.launch {
            repository.registerUser(user)
        }
    }

    // Login a user (using email):
    fun loginUser(email: String, onResult: (User?) -> Unit) {
        loggedInEmail = email
        viewModelScope.launch {
            val user = repository.loginUser(email)
            onResult(user)
        }
    }
}
```

UserViewModelFactory.kt:

```
package com.example.ia4_bakery_kp.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import com.example.ia4_bakery_kp.data.repository.UserRepository

// Factory class for UserViewModel: builds and returns a UserViewModel
when requested by ViewModelProvider
class UserViewModelFactory(private val repository: UserRepository) :
ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T { //
        if (modelClass.isAssignableFrom(UserViewModel::class.java)) {
// Check if the requested ViewModel is of type UserViewModel
            @Suppress("UNCHECKED_CAST")
            // Create and return a new instance of UserViewModel
            return UserViewModel(repository) as T
        }
        throw IllegalArgumentException("Unknown ViewModel class") //
Throw an exception if the requested ViewModel is not of the expected
type
    }
}
```

4.8. Main

MainActivity:

```
package com.example.ia4_bakery_kp

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.viewModels
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import com.example.ia4_bakery_kp.navigation.NavGraph
import com.example.ia4_bakery_kp.viewmodel.CakeViewModel

class MainActivity : ComponentActivity() {
    private val cakeViewModel: CakeViewModel by viewModels() //
Initialize 'cakeViewModel' to hold/manage shared app data across the
home, menu and order screens

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                Surface {
                    NavGraph() // Initialize the navigation graph
                }
            }
        }
    }
}
```

5. Screenshots

This video demonstrates the functionality of the search bar in the Cake Manu Screen.

Search_Use_Demo_KP.mp4

Figure 1: Sign-up Screen

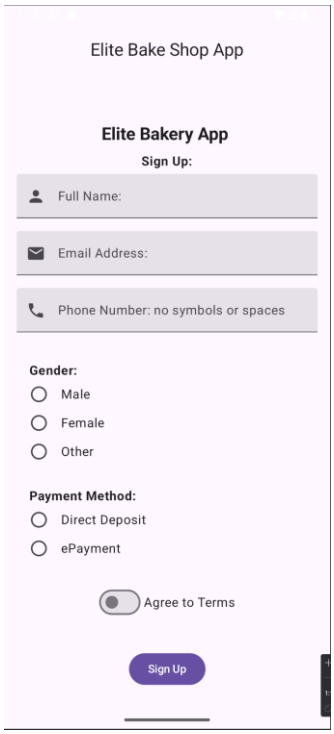


Figure 2: Error Message for Sign-up Screen

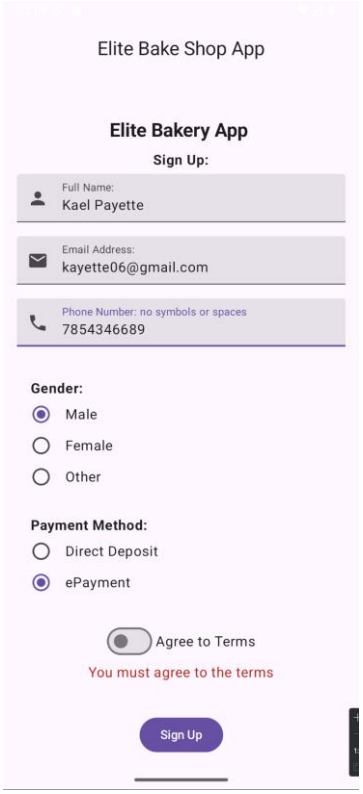


Figure 3: If Submit Form w/o Filling in All Values

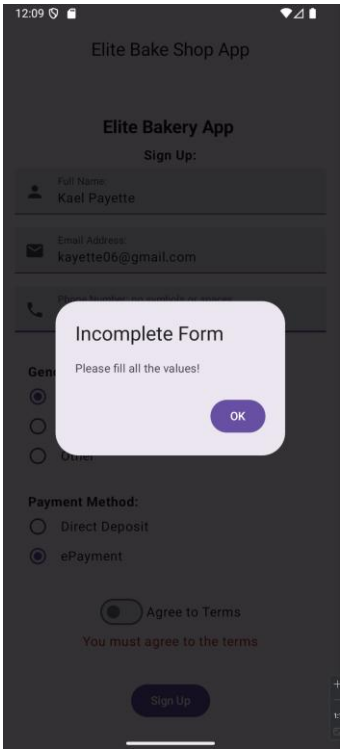


Figure 4: Log-in with Non-Registered User

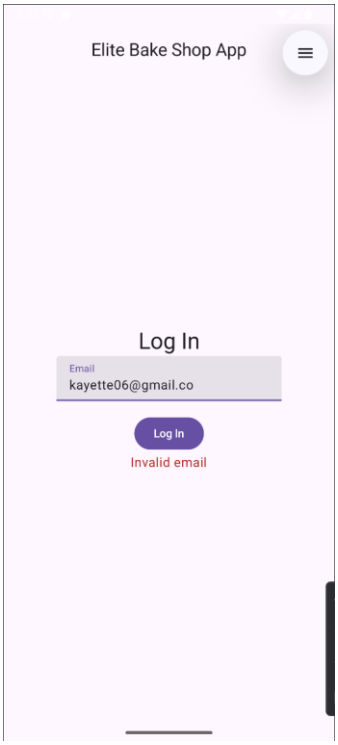


Figure 5: Log-in Screen

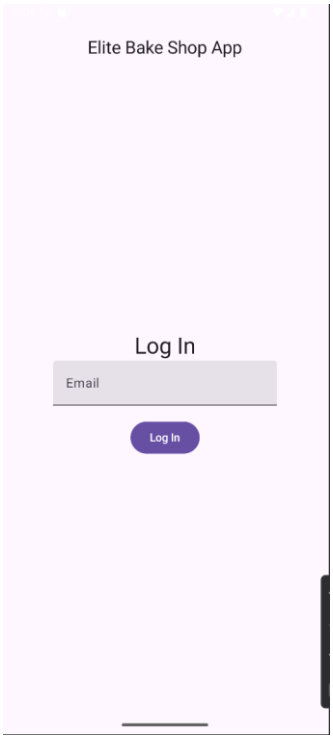


Figure 1: Home Screen

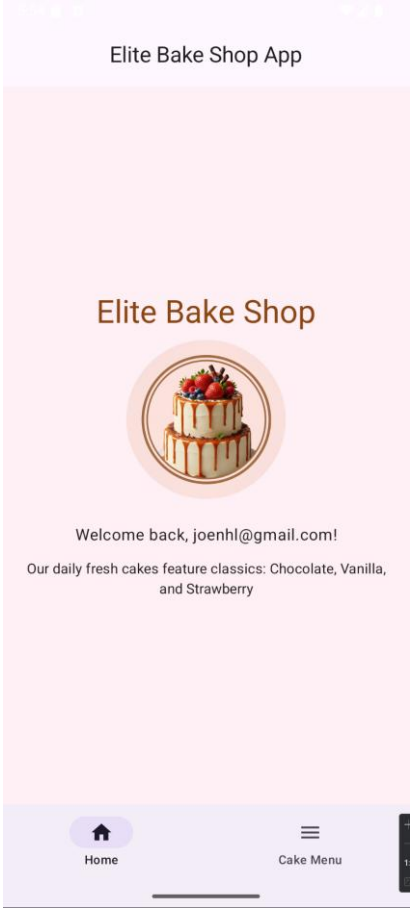


Figure 7: Cake Menu (All Items)

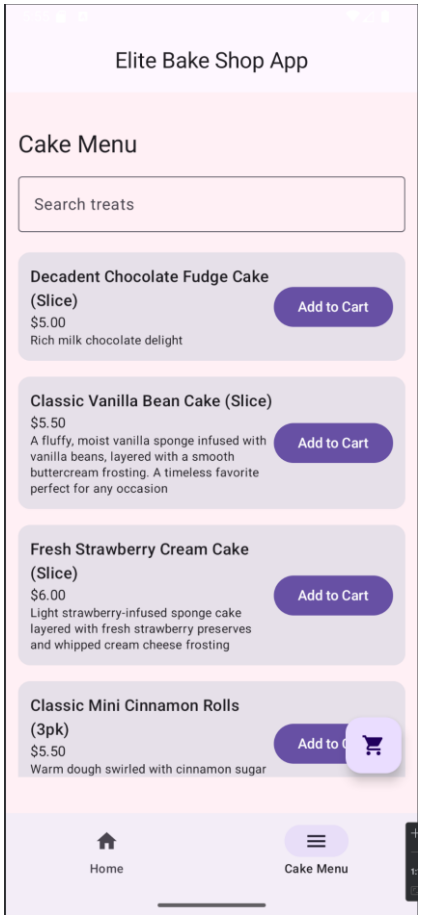


Figure 8: Items Added to Cart (Cake Menu View)

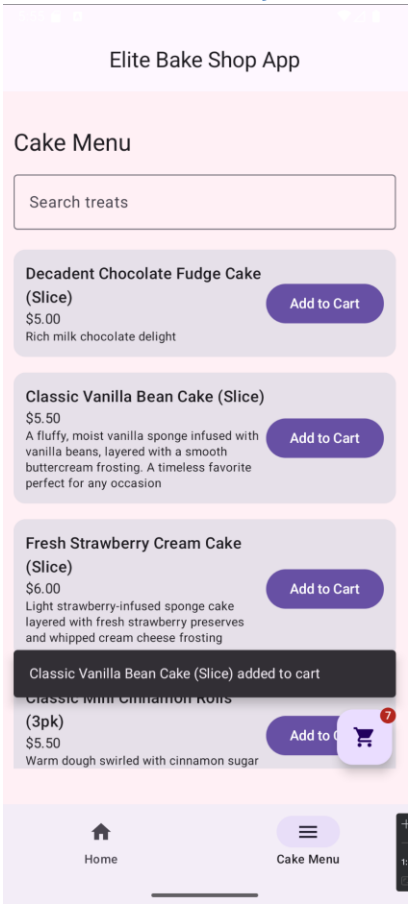


Figure 9: Items in the Cart (Cart View)

Figure 10: Order Receipt Screen

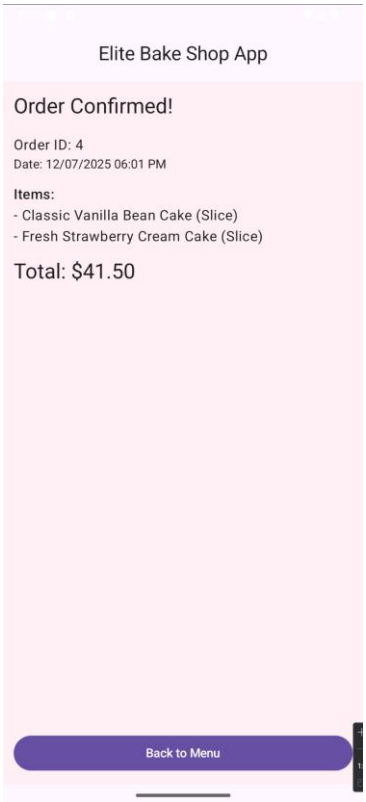
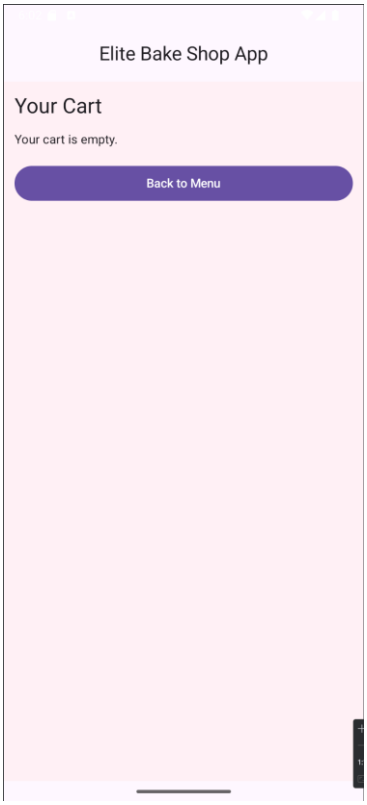


Figure 11: Empty Cart



6. Code Explanation/ Justifications

- Used SQL to make the database using Kotlin keywords
- A message saying welcome back is used whenever the email matches from login to sign up using the name entered by that user
- User table created
- Order table created
- Cake table created
- Separated each file into packages, data->dao/entity/repository | navigation | UI->home/login/menu/order/signup | viewmodel
- Cakes table created with many different types of treats
- Search function uses any word from the multi word treat names, makes it easier to find
- Would have used proper password creation and log in if had more time to implement features
- A consistent pink bakery theme was used throughout the application to make it more realistic and pleasing to look at.
- I used Gemini Ai to generate me a bakery message for the home screen as well as the pictures of: the app logo, the home page logo and every picture of each type of dessert
- 'Badgebox' was used to create the shopping cart "button" this was done as I did not want the shopping cart to be a bottom nav bar menu option as it would be confusing from a user's perspective, that is why I included the chopping cart icon 'badgebox' to represent the cart/order screen navigation as most real world applications have this, an icon clearly indicating how to navigate to the shopping cart from the menu or area where you add things to your cart.
- A single instance of the database is created and maintained for this application, aka Singleton. Thread protection is in place to make sure that all threads receive updates to the database at the same time, as well as ensure only 1 thread is creating the database, if it needs to be created.
- Because the Sign Up page's warnings take up space on the screen. The signup page is scrollable incase the messages push the sign up button below the bottom of the screen.

7. References

Android Developers. (n.d.). *Get started with Jetpack Compose*. https://developer.android.com/develop/ui/compose/documentation

GitHub Community. (2025, January 6). *Best practices for structuring Jetpack Compose applications* [Discussion #148625]. GitHub. https://github.com/orgs/community/discussions/148625

Jahani, A. (2024, May 3). *How to structure a Jetpack Compose project*. Medium. https://medium.com/@ilhom/thanks-for-the-post-can-you-recommend-any-open-source-project-that-has-followed-this-structure-1eda671d184e

Google Gemini. (n.d.). *Images and home page message generated by Gemini AI*. https://gemini.google.com

Android Developers. (n.d.). *Save data in a local database using Room*. https://developer.android.com/training/data-storage/room/

GeeksforGeeks. (2025, July 23). *Android – Data Access Object in Room Database*. https://www.geeksforgeeks.org/android/android-data-access-object-in-room-database/

Android Developers. (n.d.). *Referencing complex data using Room*. https://developer.android.com/training/data-storage/room/referencing-data

Android Developers. (n.d.). *Define data using Room entities*. https://developer.android.com/training/data-storage/room/defining-data

GeeksforGeeks. (2025, July 23). *How to use Singleton Pattern for Room Database in Android?*. https://www.geeksforgeeks.org/kotlin/how-to-use-singleton-pattern-for-room-database-in-android/

Bhadoria, D. S. (2024, April). *Jetpack Compose UI — How to use BadgeBox using Jetpack Compose Kotlin*. Medium. https://medium.com/@iamfahad1296_94867/building-dynamic-bottom-navigation-bars-with-badges-in-jetpack-compose-67405dfe8ef1

GeeksforGeeks. (2025, July 23). *Use Case Diagram – Unified Modeling Language (UML)*. https://www.geeksforgeeks.org/system-design/use-case-diagram/

GeeksforGeeks. (2025, November 17). *How to draw Entity Relationship Diagrams (ERDs)*. https://www.geeksforgeeks.org/sql/how-to-draw-entity-relationship-diagrams/